



Clean Code

دانشکده علوم مهندسی

طراح: سروش نصیری

بهار ۱۴۰۵



۱. فاصله‌گذاری و قالب‌بندی

ظاهر فیزیکی کد به اندازه محتوای آن مهم است. قالب‌بندی مناسب خوانایی را چند برابر می‌کند و نگهداری را آسان می‌سازد.

✓ تورفتگی منظم

از فاصله یا تب به‌طور یکنواخت استفاده کنید. تیم‌های حرفه‌ای معمولاً ۴ فاصله یا ۲ فاصله را استاندارد می‌کنند.

```
// خوب
for (int i = 0; i < 10; ++i) {
    for (int j = 0; j < 10; ++j) {
        std::cout << i << " ";
    }
}

// بد: نامنظم
for (int i = 0; i < 10; ++i) {
    std::cout << i << " ";
    if (i % 2 == 0) {
        std::cout << "even";
    }
}
```

✓ استفاده از خطوط خالی

بین توابع، بلوک‌های منطقی و حلقه‌های تو در تو خط خالی بگذارید تا تنفس کد حفظ شود.

```
// توصیه شده
void processData() {
    validate();

    if (condition) {
        compute();
    }

    finalize();
}
```

✓ طول خطوط

خطوط بلند را با شکستن به چند خط خواناتر کنید.

```
// طولانی و بد
std::cout << "نام کاربر: " << user.getName() << " سن: " << user.getAge() << " ایمیل: " <<
user.getEmail() << " تلفن: " << user.getPhone() << std::endl;

// خوب: شکسته شده
std::cout << "نام کاربر: " << user.getName()
    << " سن: " << user.getAge()
    << " ایمیل: " << user.getEmail()
    << " تلفن: " << user.getPhone() << std::endl;
```



۲. نام‌گذاری معنادار

نام متغیرها، توابع و کلاس‌ها باید گویای مقصود باشند. از نام‌های یک یا دو حرفی (به جز متغیرهای موقت حلقه) پرهیز کنید.

♦ استانداردهای رایج نام‌گذاری

- **camelCase**: حرف اول کوچک، کلمات بعدی با حرف بزرگ شروع می‌شوند. مثال: `calculateTotalPrice`, `userName`.
- **snake_case**: تمام حروف و کلمات با `underline` جدا می‌شوند. مثال: `calculate_total_price`, `user_name`.
- **PascalCase**: هر کلمه با حرف بزرگ. مثال: `FileProcessor`, `StudentManager`.

به دو شکل اصلی دیده می‌شود: `snake_case`

1. **lower_snake_case**: تمام حروف کوچک. مثال: `get_user_by_id`, `employee_name`.
2. **UPPER_SNAKE_CASE**: تمام حروف بزرگ. مثال: `MAX_CONNECTIONS`, `PI`.

📌 گایدلاین پروژه

در هر پروژه نرم‌افزاری، پیش از شروع کدنویسی، یک راهنمای سبک نام‌گذاری از طرف تیم یا ناظر پروژه ابلاغ می‌شود. پایبندی به این گایدلاین باعث یکپارچگی کد و کاهش سردرگمی می‌شود. فرض کنید ناظر پروژه به ما اعلام کرده است که از قواعد زیر پیروی کنیم:

- نام توابع (Functions) → `camelCase` (مثال: `calculateTotalPrice`)
- نام متغیرهای معمولی (Variables) → `lower_snake_case` (مثال: `user_age`)
- نام ثابت‌ها (Constants) → `UPPER_SNAKE_CASE` (مثال: `MAX_BUFFER_SIZE`)

📖 سبک نام‌گذاری Oracle Code Conventions (جاوا)

در استاندارد رسمی Oracle برای جاوا: نام کلاس‌ها **PascalCase**، نام متدها و متغیرها **camelCase**، و ثابت‌ها **UPPER_SNAKE_CASE** هستند. در C++ نیز معمولاً کتابخانه استاندارد از **lower_snake_case** برای توابع و متغیرها استفاده می‌کند.

📌 اعلام گایدلاین در بالای کد

برای شفافیت کامل، در تمرینات و پروژه‌های تحویلی شما باید در ابتدای فایل کد (قبل از هر خط اجرایی) با کامنت واضح اعلام کنید که هر دسته از شناسه‌ها از کدام قاعده پیروی می‌کنند. نمونه قالب استاندارد:

```
/*
 * ===== Naming Convention Guideline =====
 * Class names: PascalCase
 * Function names : camelCase
 * Variable names : lower_snake_case
 * Constant names : UPPER_SNAKE_CASE
 * =====
 */
```



۳. توابع کوچک و تک‌منظوره

قلب کد تمیز، توابعی هستند که فقط «یک کار» انجام می‌دهند و حجم کمی دارند. این اصل خوانایی، آزمون‌پذیری و نگهداری را به شدت افزایش می‌دهد.

اندازه توابع

یک تابع به ندرت باید بیش از ۲۰ خط بدنه داشته باشد (بدون احتساب خطوط خالی و براکت‌ها). بسیاری از توابع عالی زیر ۱۰ خط هستند. اگر تابعی از ۳۰-۴۰ خط عبور کرد، قاعدتاً بیش از یک کار انجام می‌دهد و باید تفکیک شود.

```
// بد - خیلی بزرگ و چند وظیفه
void handleUserData(User& user) {
    // اعتبارسنجی (خط 10)
    // نخیره در دیتابیس (خط 12)
    // ارسال ایمیل (خط 8)
    // به‌روزرسانی آمار (خط 7)
}

// خوب - هر کار در تابع مجزا
bool validateUser(const User& user);
void saveUserToDatabase(const User& user);
void sendWelcomeEmail(const User& user);
void updateStatistics(const User& user);
```

🎯 اصل Single Responsibility

یک تابع باید دقیقاً یک دلیل برای تغییر داشته باشد. اگر تابع هم اعتبارسنجی کند هم محاسبه مالیاتی انجام دهد، نقض این اصل است.

```
// نقض SR
double calculateAndValidateSalary(Employee e) { ... }

// تفکیک
bool isValidEmployee(const Employee& e);
double calculateNetSalary(const Employee& e);
```

🏠 تابع main کوتاه

تابع main فقط باید توابع سطح بالای برنامه را ترتیب دهد و جزئیات را به توابع دیگر بسپارد. حداکثر ۱۰ خط.

```
int main() {
    readConfiguration();
    initializeResources();
    runMainLoop();
    cleanupResources();
    return 0;
}
```



🔧 پیاده‌سازی متدها بیرون از کلاس

برای کاهش شلوغی تعریف کلاس و افزایش خوانایی، بدنه متدهای طولانی را بیرون از کلاس بنویسید. این کار همچنین فایل هدر را تمیز نگه می‌دارد.

```
// کلاس در فایل .hpp
class DatabaseManager {
public:
    void connect(const std::string& host, int port);
    bool executeQuery(const std::string& sql);
};

// بیرون کلاس (cpp) پیاده‌سازی در فایل
void DatabaseManager::connect(const std::string& host, int port) {
    // جزئیات اتصال
}

bool DatabaseManager::executeQuery(const std::string& sql) {
    // اجرای کوئری
}
```

🔧 Constructor و Destructor (سازنده و مخرب)

constructor برای مقداردهی اولیه و تأمین نیازمندی‌های اولیه شیء استفاده می‌شود. destructor برای آزادسازی منابع (مانند حافظه پویا، فایل باز) ضروری است. حتماً constructor و destructor را پیاده‌سازی کنید.

```
class FileHandler {
private:
    FILE* file_handle;
public:
    FileHandler(const char* filename) { // constructor
        file_handle = fopen(filename, "r");
        if (!file_handle) throw std::runtime_error("cannot open");
    }

    ~FileHandler() { // destructor
        if (file_handle) fclose(file_handle);
    }
};
```



🔒 توابع const (متدهایی که شیء را تغییر نمی‌دهند)

هر متدی که وضعیت شیء را تغییر نمی‌دهد (getterها، توابع محاسباتی فقط خواندنی) باید با پسوند `const` اعلام شود. این کار قصد برنامه‌نویس را مشخص می‌کند و از تغییر تصادفی داده‌ها جلوگیری می‌کند.

```
class Person {
private:
    int age;
    std::string name;
public:
    int getAge() const { return age; } // const: تغییر نمی‌کند
    std::string getName() const { return name; }
    void setAge(int new_age) { age = new_age; } // const غیر (تغییر می‌دهد)
};
```

📌 نکته کلیدی

این عمل به بهینه سازی کامپایلر هم به مقدار خوبی کمک میکند.

۴. عدم استفاده از Magic Number

اعداد ثابت و بدون توضیح (جادویی) خوانایی و نگهداری را سخت می‌کنند. به جای آن از ثابت‌های نمادین (`const` یا `enum`) استفاده کنید.

```
// بد: 37 و 86,400 یعنی چی؟
if (temperature > 37) {
    sendAlert();
}
int delay = 86400;

// خوب: ثابت معنادار
const double HUMAN_FEVER_THRESHOLD = 37.0;
const int SECONDS_PER_DAY = 24 * 60 * 60; // 86400

if (temperature > HUMAN_FEVER_THRESHOLD) {
    sendAlert();
}
int delay = SECONDS_PER_DAY;
```

📌 مزایای حذف Magic Number

- تغییر در یک نقطه (مرکزی) کل کد را به‌روز می‌کند.
- معنی عدد برای خواننده مشخص است.
- خطای تایپی کاهش می‌یابد.



۵. DRY (Don't Repeat Yourself)

از تکرار کد به هر قیمتی پرهیز کنید. هر قطعه کد باید یکبار و فقط یکبار در سیستم پیاده‌سازی شود.

مثال بازنویسی تکراری

```
// بد: تکرار محاسبه تخفیف در چند جا
double price1 = basePrice * 0.9;
double price2 = anotherBase * 0.9;

// خوب: تابع کمکی
double applyDiscount(double basePrice, double discountRate) {
    return basePrice * discountRate;
}

const double STANDARD_DISCOUNT = 0.9;
double price1 = applyDiscount(basePrice, STANDARD_DISCOUNT);
```

همچنین از کپی‌کردن بلوک‌های شرطی یا حلقه‌ها خودداری کنید. در صورت نیاز، منطق تکراری را در یک تابع پیاده کنید.

۶. کامنت‌گذاری مفید

کد خوب خودش را توضیح می‌دهد. کامنت‌ها باید «چرایی» را بیان کنند، نه «چگونگی» را. از کامنت‌های بدیهی و توضیح خط به خط پرهیز کنید.

❌ کامنت بد

```
int counter = 0; // شمارنده را صفر کن
// حلقه برای چاپ اعداد
for (int i = 0; i < 10; i++) {
    cout << i; // چاپ i
}
```

✅ کامنت خوب (توضیح منطق پیچیده یا دلیل تصمیم)

```
// از الگوریتم اقلیدس برای کاهش زمان محاسبه ب.م.م استفاده شده
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

نکته: به جای کامنت‌های توضیح‌دهنده، نام‌های بهتری برای متغیرها و توابع انتخاب کنید. مثلاً به جای

بررسی اعتبار کدپستی // ، تابع isPostalCodeValid() بنویسید.